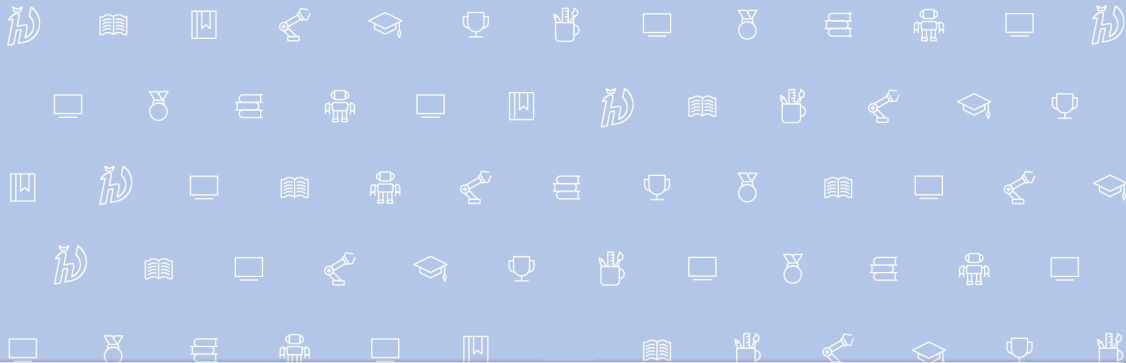




Artificial Intelligence Software, Project 2

Inference in the Loop: AI-Driven Robot Control with ROS2



Center for
Cognitive
Modeling



Short Description

The goal of this project is to build an **AI-driven robot control system** by integrating a trained ML model with a ROS2 pipeline using **ONNX** as the inference format.

The full pipeline:

1. Define an AI task and a robot action mapping
 2. Train or fine-tune a model
 3. Export the model to **ONNX**
 4. Deploy inference inside a **ROS2 node**
 5. Control a robot via ROS topics based on model output.
 6. Monitor live inference in a **Gradio** panel
 7. Demonstrate in **Gazebo** or on a real robot
- Students are free to choose any modalities like: CV, audio, NLP, etc.

Reusing a model from **Project 1** is encouraged but not required.



Example Project

Gesture-Controlled Differential-Drive Robot

A YOLO classification model recognize **hand gestures** from a webcam. Each gesture class is mapped to a robot motion command:

Gesture classes (example):

- forward → move forward
- backward → move backward
- turn_left → rotate left
- turn_right → rotate right
- stop → zero velocity

The robot publishes velocity commands to `/cmd_vel`. A **Gradio** panel displays the live predicted class and confidence score.



Step 1

Short description: Define the AI task your system will solve and specify how the model output maps to robot actions.

Artifacts: Report section describing the task design.

Requirements

- Describe the input modality (camera, microphone, etc.)
- Specify the model output: class labels, bounding boxes, regression values, etc.
- Define the complete **class-to-action mapping**
(e.g. class forward $\rightarrow$ linear.x = 0.3)
- Justify why the chosen task is suitable for robot control
- The system must support at least **3 distinct robot actions**



Step 2

Short description: Obtain a trained AI model that produces the outputs defined in Step 1 and is exportable to ONNX.

Artifacts: Model checkpoint (or downloaded weights), evaluation results, report section.

Requirements

- Use any ONNX-exportable framework (Ultralytics YOLO, PyTorch, HuggingFace, etc.)
- **Fine-tuning is not required** — you may use a pre-trained model directly if it already solves your task
- If the available pre-trained models do not fit, fine-tune on a suitable dataset
- Evaluate the chosen model with appropriate metrics (accuracy, mAP, F1, etc.)
- Describe the model selection or training process in the report
- ClearML experiment tracking is **recommended** if training is performed



Step 3

Short description: Export the trained model to ONNX format and validate that inference produces results consistent with the original framework.

Artifacts: .onnx model file, validation script, report section with comparison results.

Requirements

- Export the model to ONNX using the framework's export API
- Document: input/output tensor names and shapes, opset version, dynamic axes if used
- Run a validation script: pass the same input through both the original model and onnxruntime
- Verify outputs match within floating-point tolerance
- Measure and report inference latency for the ONNX model on your hardware



Step 4

Short description: Design and document the ROS2 system architecture for your project.

Artifacts: Architecture description in the report with all node and topic names defined.

Requirements

- You choose how inference integrates with ROS2:
 - **Outside ROS2** — a standalone script reads the sensor directly and publishes the result to a ROS2 topic
 - **Inside ROS2** — a dedicated ROS2 node subscribes to a sensor topic, runs ONNX, and publishes the result

Either way, the ROS2 side must include at minimum: a **result topic** carrying the model output (e.g. /model_result); a **controller node** that subscribes to the result and commands the robot

- List all ROS2 topics: name, message type, publisher, subscriber
- Justify your integration choice in the report
- Describe the complete data flow: sensor → inference → ROS2 topic → robot actuator



Step 5

Short description: Implement the inference component that runs the ONNX model and publishes the result to a ROS2 topic.

Artifacts: Inference script or ROS2 node source code.

Requirements

- Implement according to the architecture chosen in Step 4:
 - **Outside ROS2:** a Python script reads sensor data directly (e.g. via OpenCV, PyAudio), runs ONNX inference, and publishes to `/model_result` via `roscpp`
 - **Inside ROS2:** a ROS2 node subscribes to a sensor topic, runs ONNX inference, and publishes to `/model_result`
- Load the ONNX model using `onnxruntime` at startup
- Preprocess input data to match the model's expected input shape
- Published message must contain: predicted class name and confidence score



Step 6

Short description: Implement a ROS2 controller node that subscribes to the model result and commands the robot accordingly.

Artifacts: ROS2 package containing the controller node source code.

Requirements

- Subscribe to `/model_result` published by the inference script
- Map each predicted class or value to an appropriate ROS2 command message.

The command type depends on the robot:

- mobile robot: `geometry_msgs/Twist` on `/cmd_vel`
 - manipulator: `trajectory_msgs/JointTrajectory`
 - custom actuator: any suitable message type
- All action-to-command mappings must be explicitly defined and documented
- Include a configurable **confidence threshold** — send a safe/neutral command if confidence is below it



Step 7

Short description: Build a Gradio application that subscribes to the inference result topic and displays a live monitoring panel.

Artifacts: Python source code for the Gradio monitoring app.

Requirements

- The panel must display in real time:
 - current predicted class label
 - confidence score
 - optionally: the latest input frame
- Subscribe to `/model_result` via `rclpy` or a shared queue
- Use appropriate Gradio components (`gr.Label`, `gr.Image`, etc.)
- The panel must update continuously without manual refresh
- The app runs as a **separate process** alongside the ROS2 nodes



Step 8

Short description: Demonstrate the complete integrated system with the robot responding to live model inference while the Gradio panel monitors inference output.

Artifacts: Recorded demo video; report section describing the demo setup.

Requirements

- All components must run simultaneously and be visible in the recording:
 - live sensor input
 - robot reacting in **Gazebo simulation** or on a **real robot**
 - **Gradio panel** updating in real time
- The video must demonstrate at least **3 distinct robot actions** triggered by model output
- Describe the simulation world or real robot setup in the report



Submission

You must submit a **report** and a working code repository covering the full pipeline.

The report must include:

- task design and action mapping (Step 1)
- model training process and results (Step 2)
- ONNX export settings and validation (Step 3)
- ROS2 architecture description (Step 4)
- description of all implemented nodes (Steps 5–6)
- Gradio panel description and screenshots (Step 7)
- demo setup and observations (Step 8)
- discussion of difficulties and limitations

Required artifacts:

- Git repository with all source code
- trained .onnx model file
- demo video (robot + Gradio panel)
- PDF report

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521